CONFIDENTIAL · PENETRATION TEST REPORT

National Institute of Business Management (NIBM)

in academic partnership with Coventry University

BSc (Hons) Ethical Hacking and Network Security

NB6013CEM — Penetration Testing · Coursework Assessment No. 1 (100 marks)

Penetration Testing Engagement Report

SecureBank Online Banking Application

Parts A & B — Consolidated Engagement Plan and Execution Report

Student Name	Abdullah M A
Coventry Index Number	16124507
NIBM Index Number	COBSCEHNS24.2P-017
Assigned Application	System 03 — SecureBank (index 017 → 017 MOD 3 = 2)
Programme / Batch	BSc (Hons) Ethical Hacking and Network Security · 24.2P
Module	NB6013CEM — Penetration Testing
Module Lecturer	Kamal Sampath Upasena
Assessment Mode / Type	Individual · Consolidated report (Parts A & B)
Submission	One consolidated PDF, submitted to the LMS

Contents

Contents 2
Part A — Engagement Plan and Scope 3
A1. Introduction and Organisational Background 3
A2. Engagement Objectives 3
A3. Scope and Boundaries 3
A4. Penetration Testing Methodology 4
A5. Legal, Ethical and Contractual Considerations 4
A6. Rules of Engagement and Deconfliction 5
A7. Risk Management and Engagement Schedule 5
Part B — Execution and Penetration Test Report 7
B1. Executive Summary 7
B2. Technical Environment Overview 7
B3. Reconnaissance and Enumeration 8
B4. Attack Surface Analysis 10
B5. Vulnerability Identification and Validation 11
B6. Exploitation and Proof of Concept 19
B7. Post-Exploitation and Attack-Path Analysis 20
B8. Vulnerability Risk Assessment 21
B9. Business Impact Analysis 22
B10. Remediation Recommendations 22
B11. Conclusion 23
References 23
Appendix A — Evidence Index 24
Appendix B — Tools Used 24

Part A — Engagement Plan and Scope

Prepared before testing began. This part assesses ILO1 — planning and scoping a penetration test within a defined legal, ethical and contractual frame.

A1. Introduction and Organisational Background

SecureBank Ltd is a small retail bank that provides online banking to personal customers. Through its web application, customers open and view accounts, move money between accounts, review their transaction activity, and contact the bank through a support messaging feature. Behind the customer-facing site, support and operations staff use additional functions to handle customer queries and to view account information across the bank.

Because the application processes highly sensitive information — customer identities, account numbers, balances, transaction histories and private messages — and directly controls access to customer funds, its security is business-critical. SecureBank Ltd engaged the author as an independent, authorised penetration tester to assess the security posture of the application. The engagement was carried out under written authorisation, against a controlled test deployment, and within an agreed scope. The objective was not merely to find bugs, but to understand how weaknesses arise, how they could be exploited, what they would mean for the business, and how they should be remediated.

A2. Engagement Objectives

The engagement had both business and technical objectives. The business objectives were to protect customer funds and personal data, to maintain customer trust, and to support the bank in meeting its legal and regulatory obligations. The technical objectives were to evaluate the application's confidentiality, integrity and availability, with particular attention to authentication, authorisation (access control), session management, input handling, API security and business-logic controls.

A successful engagement would identify the application's significant weaknesses, validate them with reproducible evidence, demonstrate their exploitation safely within the test environment, and provide clear, prioritised recommendations the bank could act on.

A3. Scope and Boundaries

The engagement was limited to the SecureBank application deployed in the authorised test environment. The boundaries are set out below.

Item	Detail
Target application	SecureBank online banking web application (System 03)
Authorised host / port	127.0.0.1:8082 (nginx) — local Docker lab only
Backend	Go (net/http) REST API; React single-page frontend
In scope	Web UI, /api/* endpoints, authentication, authorisation, session handling, support messaging, business logic
Out of scope	Any other host; other lab applications; other students' environments; the OS / Docker host; denial-of-service and destructive testing

The testing was conducted as a grey-box assessment: the author held a standard customer account and could register further customer accounts, but had no source-code access beyond the publicly served JavaScript bundle and no administrative credentials. All testing used synthetic data in the local lab only and was time-boxed. Only the assigned application was tested, and only within the authorised environment.

A4. Penetration Testing Methodology

A structured methodology was followed so that the assessment would be systematic, repeatable and evidence-based, rather than a collection of isolated tests. The approach draws on recognised standards: the Penetration Testing Execution Standard (PTES) for the overall phases, the OWASP Web Security Testing Guide (WSTG) and OWASP API Security Top 10 for the practical web and API tests, NIST SP 800-115 for the testing process, and MITRE ATT&CK for reasoning about the attack path. The phases and the evidence each produced are summarised below.

Phase	Framework reference	Evidence it produced
Pre-engagement & scoping	PTES; NIST SP 800-115	Scope and Rules of Engagement (Part A)
Reconnaissance & enumeration	PTES; OWASP WSTG	nmap, nikto, JS bundle analysis
Attack-surface mapping	OWASP WSTG; API Top 10	Endpoint map from bank_bundle.js
Vulnerability identification	OWASP WSTG; OWASP Top 10	Burp / curl request & response
Validation	OWASP WSTG	Reproduced requests / responses
Controlled exploitation	PTES	Burp Repeater / Intruder PoC
Post-exploitation & attack path	MITRE ATT&CK	Attack-path diagram
Risk assessment & reporting	CVSS v3.1; NIST	CVSS scores, this report

In line with professional practice and the brief, automated tools were used only to assist discovery; every finding was interpreted and validated manually, and conclusions were drawn from reproducible evidence rather than from scanner output alone. The main tools were nmap (service discovery), nikto (web-server checks), Burp Suite (interception, manual testing with Repeater, and brute-force testing with Intruder), curl (raw HTTP requests), sqlmap (SQL-injection testing), the browser developer tools (traffic, cookies and the console), and standard command-line utilities for analysing the JavaScript bundle. Each tool was chosen to fit a specific phase of the methodology.

A5. Legal, Ethical and Contractual Considerations

Penetration testing involves actions that would be unlawful without authorisation, so the engagement was conducted within a clear legal and ethical framework. Testing was performed only with written authorisation and only within the agreed scope; acting outside that scope, or without permission, could constitute an offence under the Computer Misuse Act 1990, which criminalises unauthorised access to computer material.

Because the application handles personal and financial data, the author treated all data in line with the principles of the UK GDPR and the Data Protection Act 2018; only synthetic data provided in the test environment was used, and no real personal data was accessed. The author maintained confidentiality over the findings, carried out no destructive testing, took care to protect the availability of the system, handled and retained evidence securely, and disclosed findings responsibly to the system owner. These considerations ensure that the value of the test is not undermined by unlawful or unethical conduct.

A6. Rules of Engagement and Deconfliction

The Rules of Engagement (RoE) agreed with the client are summarised below.

RoE item	Agreed position
Authorised environment	SecureBank on 127.0.0.1:8082 only
Permitted activities	Recon, enumeration, manual testing, controlled exploitation of confirmed issues
Prohibited activities	DoS, data destruction, testing other hosts / apps, use of real data
Testing schedule	Agreed window (see A7)
Communication & escalation	Agreed contact; notify lecturer of unexpected behaviour
Stop conditions	Unexpected disruption, access beyond scope, instability
Evidence handling	Stored securely, redacted, retained only as needed

In addition, a deconfliction process was agreed so that the author's authorised activity could be distinguished from a genuine attack: testing was conducted from an identified source within the agreed window, the system owner was aware testing was taking place, and any unexpected disruption, instability or sign of a real third-party attack would be reported immediately to the module lecturer (acting as the system owner), with testing paused until the situation was clarified.

A7. Risk Management and Engagement Schedule

Conducting a penetration test carries its own risks to the target environment. The main risks and the mitigations applied are shown below, followed by the planned schedule of activities and deliverables.

Engagement risk	Mitigation
Application unavailability	Non-destructive testing; isolated lab; back off on instability
Accidental data modification	Read-focused testing; own test account; reset via lab script after preserving evidence
Account lockouts	Use own accounts; limit automated attempts
Excessive resource use	Throttle automated tools
Exposure of sensitive data	Synthetic data only; redact evidence
Impact on other students	Test only own local lab instance

Phase	Est. duration	Deliverable
Planning & scoping	0.5 day	Part A plan
Recon & enumeration	0.5 day	Attack-surface map
Testing & validation	1 day	Validated findings
Exploitation & analysis	1 day	PoCs, attack path
Reporting	1 day	This report

Part B — Execution and Penetration Test Report

Assesses ILO2–ILO5. Confirmed findings are distinguished from unverified ones throughout.

B1. Executive Summary

SecureBank Ltd commissioned an authorised penetration test of its online banking application to assess whether weaknesses in the application could expose customer information or allow unauthorised actions. The assessment was carried out against a controlled test deployment, within an agreed scope, using recognised testing methods.

The application was found to have several significant security weaknesses. Five vulnerabilities were confirmed and demonstrated in practice; a sixth area (SQL injection on the login) was tested and found not to be exploitable. The most serious issue is that access control is enforced in the user's browser but not on the server. As a result, an ordinary logged-in customer can retrieve the account details and balances of every customer in the bank, and can read individual customers' accounts by changing a number in a web address.

In addition, the login does not limit password guessing, so an attacker can brute-force weak passwords and take over customer accounts; this was demonstrated by recovering a test account's password. The customer support feature also stores and runs attacker-supplied script (stored cross-site scripting), which could be used to attack staff who read customer messages. Finally, the server reveals version information, is missing standard security headers, and runs over unencrypted HTTP.

Confirmed	High severity	Medium severity	Unverified	Top business risk
5	1	4	1	**Full disclosure of all customer financial data**

Taken together, these weaknesses would allow an attacker to breach the confidentiality of all customer financial data and to compromise individual accounts. For a bank, this represents a severe risk to customers, to regulatory compliance (UK GDPR) and to reputation. The priority recommendations are to enforce authorisation on the server for every request, add anti-automation controls and a strong password policy to the login, encode all user-supplied output, and correct the server configuration (including enabling HTTPS). None of the issues require sophisticated tools; they stem from missing server-side checks and insecure defaults, and are straightforward to remediate.

B2. Technical Environment Overview

The application is a single-page web application with a React front-end served to the browser as a compiled JavaScript bundle (bank_bundle.js). Requests from the browser are handled by an nginx reverse proxy on 127.0.0.1:8082, which forwards API calls under /api/ to a back-end written in Go (observed on port 36279). Sessions are maintained using a JSON Web Token (JWT) stored in a cookie; the cookie is marked HttpOnly and SameSite=Lax, which are appropriate settings. The application is served over plain HTTP. Figure 1 shows the architecture and the location of the central weakness: authorisation decisions are made in the React bundle rather than on the server.

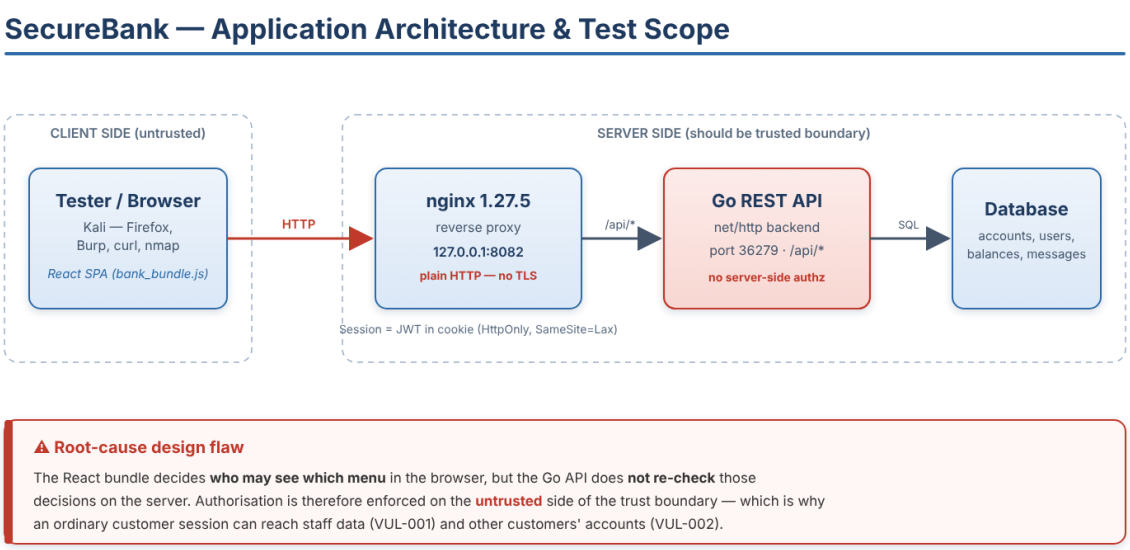


Figure 1 — SecureBank architecture and test scope. Authorisation is enforced in the client-side React bundle, not on the Go API.

Component	Detail
Frontend	React single-page app (bank_bundle.js)
Reverse proxy	nginx 1.27.5 on 127.0.0.1:8082 (HTTP)
Backend API	Go (net/http), observed on port 36279; routes under /api/*
Session	JWT in a cookie (HttpOnly, SameSite=Lax)
Data	Customer accounts, balances, users, support messages (synthetic)

B3. Reconnaissance and Enumeration

Reconnaissance was used to understand the technologies in use and to map the application's attack surface before any testing. A service scan with nmap (Figure 2) identified nginx 1.27.5 on port 8082 and a Go net/http service on port 36279. A web-server scan with nikto (Figure 3) confirmed the outdated nginx version and reported several missing security headers.

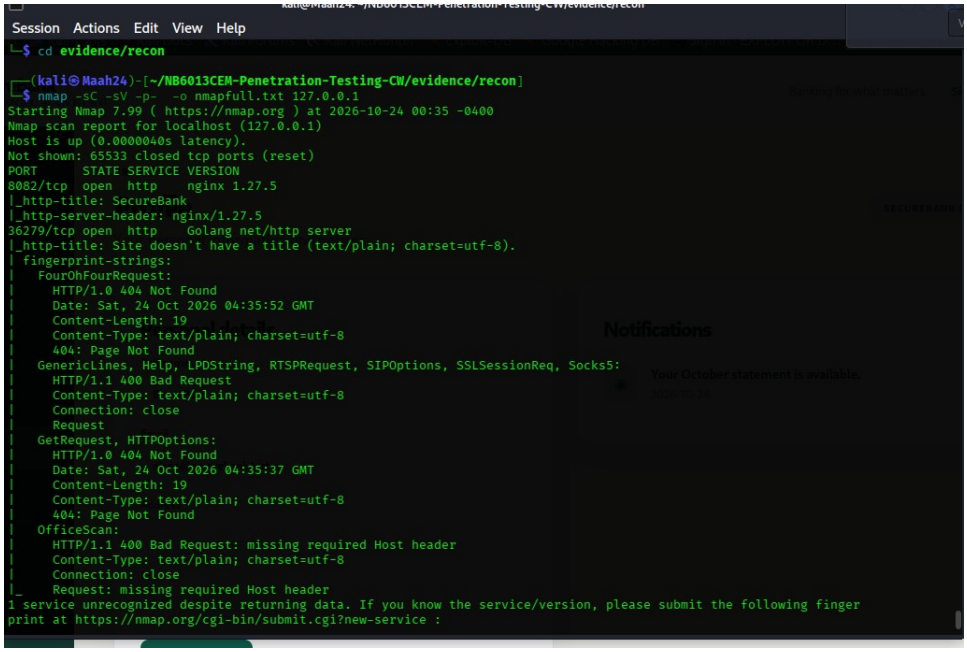


Figure 2 — nmap service scan: nginx 1.27.5 on 8082 and a Go service on 36279.

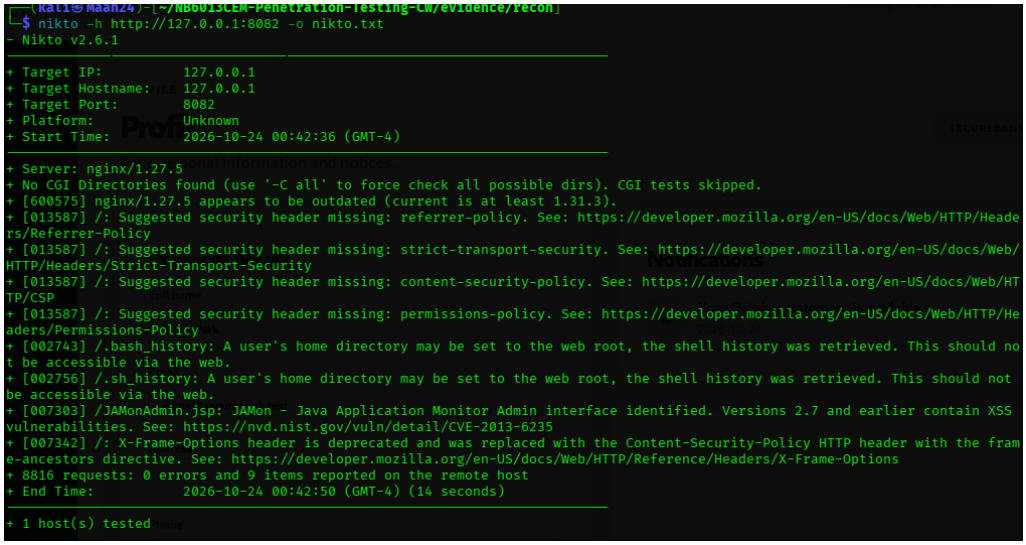


Figure 3 — nikto scan: outdated nginx and missing security headers.

To understand the application's functionality, the author retrieved the front-end entry page and located the compiled JavaScript bundle it references (Figure 4), then downloaded the bundle for offline analysis (Figure 5). Searching the bundle (Figure 6) revealed the complete set of API routes used by the application — including /api/accounts, /api/transfers, /api/support, /api/notifications and, importantly, a staff-oriented /api/operations route — and showed that the Operations menu is displayed only to users whose role is support or admin.

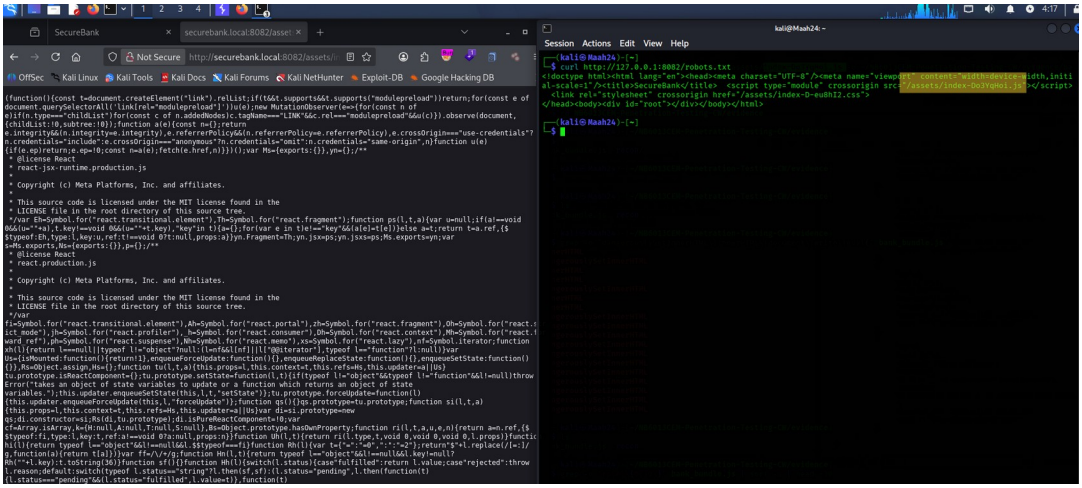


Figure 4 — Locating the React bundle referenced by the entry page.

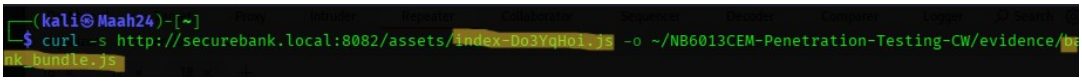


Figure 5 — Downloading bank_bundle.js for offline analysis.

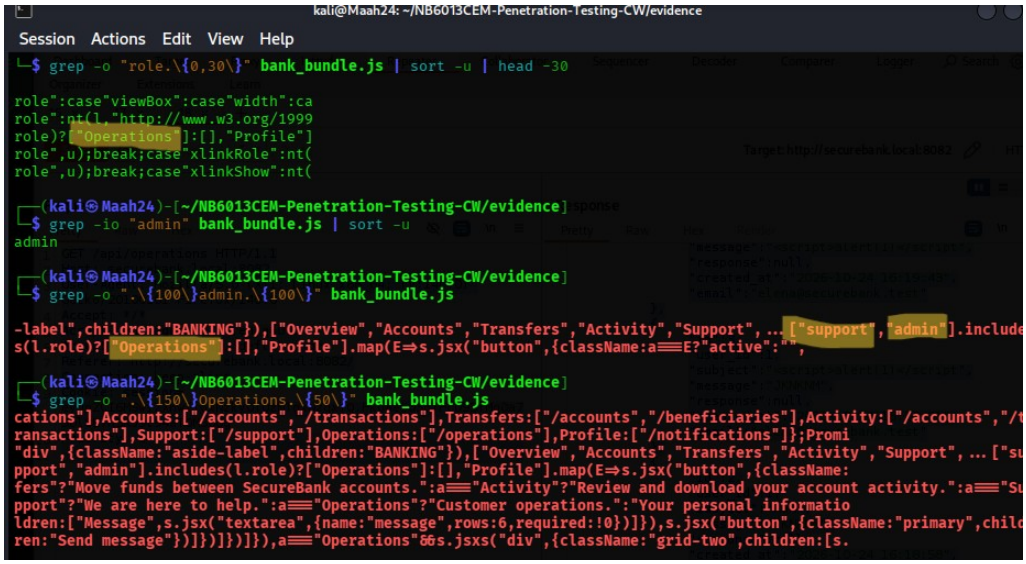


Figure 6 — API routes and client-side role checks found in the bundle.

This client-side role check became the starting point for the access-control testing that follows. Reconnaissance therefore did more than list services: it exposed hidden functionality and the design decision (client-side authorisation) that underlies the most serious findings.

B4. Attack Surface Analysis

The reconnaissance above defines the application's attack surface: a public login, a set of authenticated API endpoints for accounts, transfers, activity and notifications, a customer support messaging feature, and a staff-oriented operations endpoint. Because the application handles money and personal financial data, testing prioritised the areas where a failure would have the greatest impact: authentication (the login), authorisation (access to accounts and to the operations function), and the handling of user-supplied input (the support feature). The endpoints examined are listed below.

Area	Endpoint	Notes
Auth	`/api/auth/login`	Login — brute-force tested
Accounts	`/api/accounts/{id}`	Per-object access — IDOR tested
Transfers	`/api/transfers`	Money movement
Activity	`/api/accounts/transactions`	Statement / activity
Support	`/api/support`	Messaging — XSS tested
Operations	`/api/operations`	Staff-only in UI — access-control tested
Notifications	`/api/notifications`	Profile / notifications

B5. Vulnerability Identification and Validation

The six findings are detailed below, each with its facts, how it was discovered and validated, and the supporting evidence. Confirmed findings are reproduced from the author's own testing; VUL-006 is recorded honestly as tested but not confirmed.

ID	Vulnerability	OWASP	Status
VUL-001	Broken function-level access control (/api/operations)	A01:2021 Broken Access Control	**Confirmed**
VUL-002	IDOR / Broken Object-Level Authorisation (/api/accounts/{id})	A01:2021 Broken Access Control	**Confirmed**
VUL-003	Broken authentication: no rate limiting / lockout	A07:2021 Identification & Authentication Failures	**Confirmed**
VUL-004	Stored Cross-Site Scripting (support messages)	A03:2021 Injection (XSS)	**Confirmed**
VUL-005	Security misconfiguration (version disclosure, missing headers, client-side gating)	A05:2021 Security Misconfiguration	**Confirmed**
VUL-006	SQL injection on login (tested, not confirmed)	A03:2021 Injection	**Unverified**

VUL-001 — Broken function-level access control (/api/operations)

Field	Detail
Finding ID	VUL-001
Affected component	GET /api/operations (Go REST API)
OWASP	A01:2021 Broken Access Control; API5:2023 Broken Function-Level Authorisation
CWE	CWE-862 Missing Authorization
Validation status	Confirmed — exploited

During reconnaissance the author examined the application's compiled JavaScript bundle (bank_bundle.js). The bundle revealed the full set of API routes, including /api/operations, and showed that the Operations

area of the interface is displayed only to users whose role is support or admin (the code checks whether the role is in the list [support, admin] before showing the Operations menu). This is a client-side control: the decision about who may use the operations feature is made in the browser.

Testing confirmed that the server does not enforce the same restriction. Logged in as an ordinary customer, the author sent a direct request to GET /api/operations and the server responded 200 OK with a JSON body containing every customer's accounts (account numbers, types and balances) together with all support messages. The same data was obtained from the command line using curl with the customer's session cookie. By contrast, a request to /api/admin/users returned 404, showing that some paths are restricted while the operations path is not. The control that should restrict this staff-only function exists only in the front-end and is therefore absent where it matters.

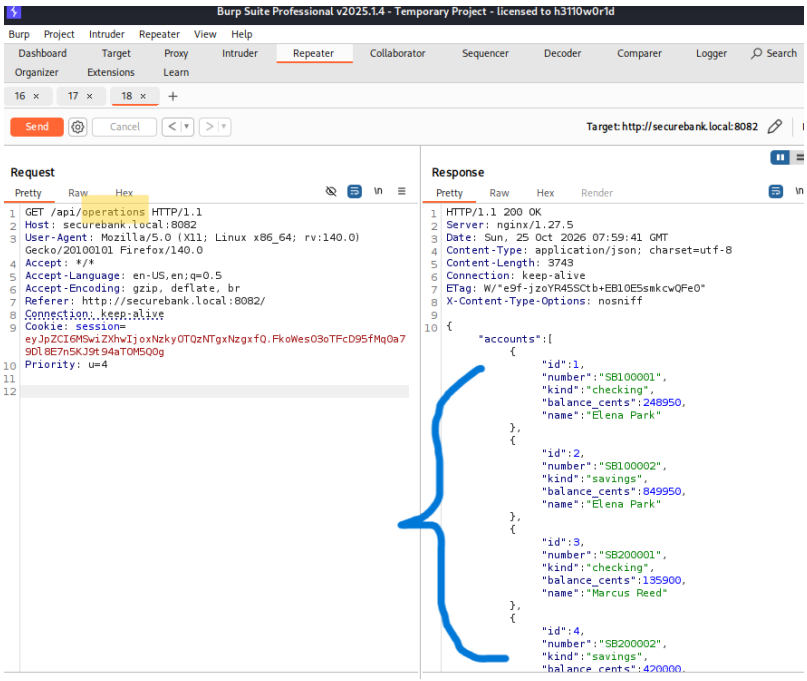


Figure 7 — GET /api/operations returns all customers' accounts (HTTP 200).

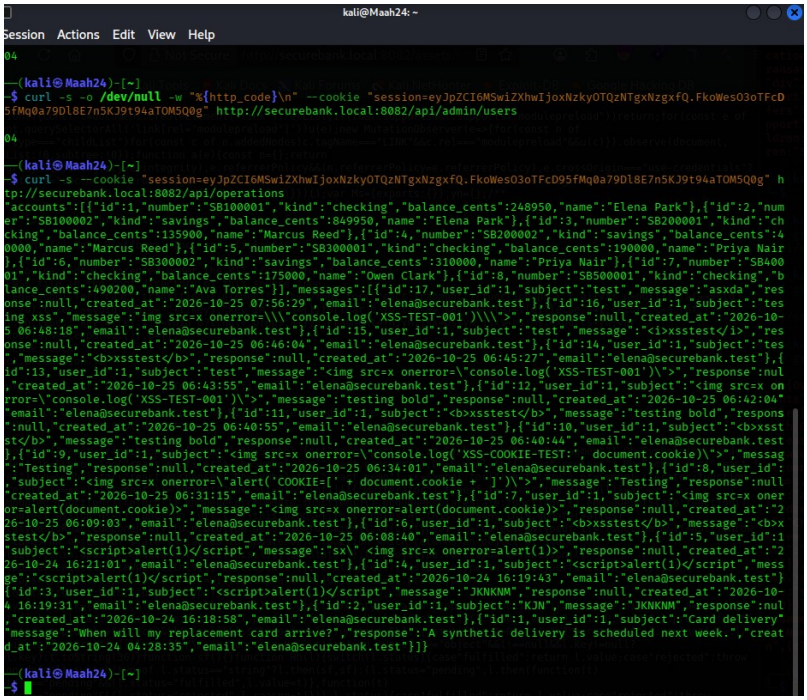


Figure 8 — The same data retrieved with curl using a customer session cookie.

VUL-002 — IDOR / Broken Object-Level Authorisation (/api/accounts/{id})

Field	Detail
Finding ID	VUL-002
Affected component	GET /api/accounts/{id}
OWASP	A01:2021 Broken Access Control; API1:2023 BOLA
CWE	CWE-639 Authorization Bypass Through User-Controlled Key
Validation status	Confirmed — exploited

The application exposes individual accounts through the pattern /api/accounts/{id}, where {id} is a numeric identifier in the URL. The author observed that the identifier could simply be changed to another number, and that the server returned the requested account without checking that it belonged to the logged-in customer. Requesting /api/accounts/2 returned an account whose user_id was 1, while /api/accounts/3 returned an account whose user_id was 2 — a different customer. Because the object reference is supplied by the user and is not authorised against the session, any authenticated customer can read other customers' account records by iterating through identifiers. This is an Insecure Direct Object Reference, also described as Broken Object-Level Authorisation.

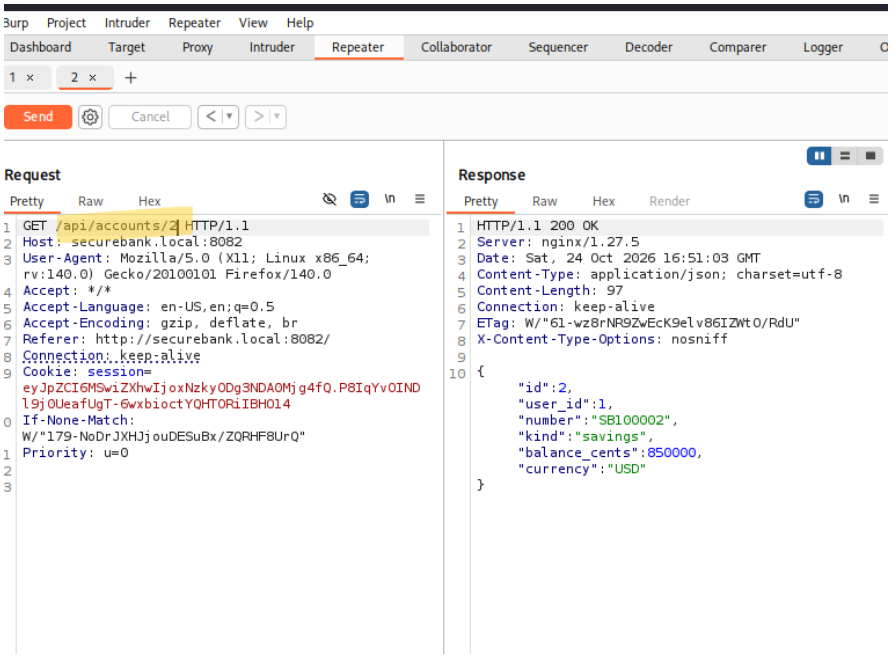


Figure 9 — IDOR: GET /api/accounts/2 (user_id 1).

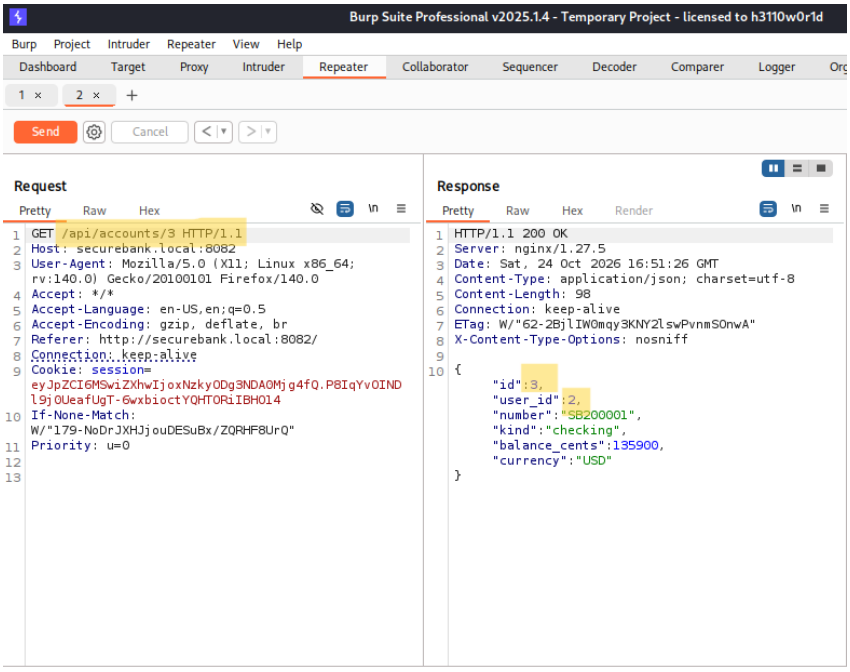


Figure 10 — IDOR: GET /api/accounts/3 returns another customer's account (user_id 2).

VUL-003 — Broken authentication: no rate limiting / lockout (brute force)

Field	Detail
Finding ID	VUL-003
Affected component	POST /api/auth/login
OWASP	A07:2021 Identification and Authentication Failures
CWE	CWE-307 Improper Restriction of Excessive Authentication Attempts

Field	Detail
Validation status	Confirmed — exploited

The login endpoint, POST /api/auth/login, accepts password attempts without any limit, delay, lockout or CAPTCHA, so an attacker can submit as many guesses as they like. To confirm this, the author used Burp Intruder with a short list of common and likely passwords against a known account. The application returned HTTP 401 for every incorrect guess and HTTP 200 for the correct one, so the responses themselves reveal when a guess succeeds. After a small number of attempts the password Welcome123! was recovered, and no request was blocked or throttled during the attack. The underlying weakness is the absence of anti-automation controls on authentication; combined with a weak or default password, this leads directly to account takeover.

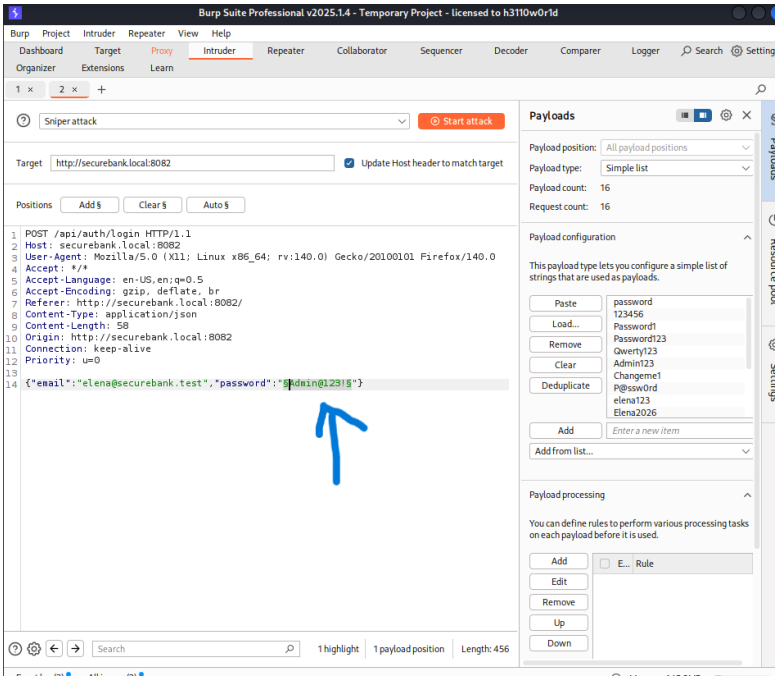


Figure 11 — Burp Intruder brute-force setup against /api/auth/login.

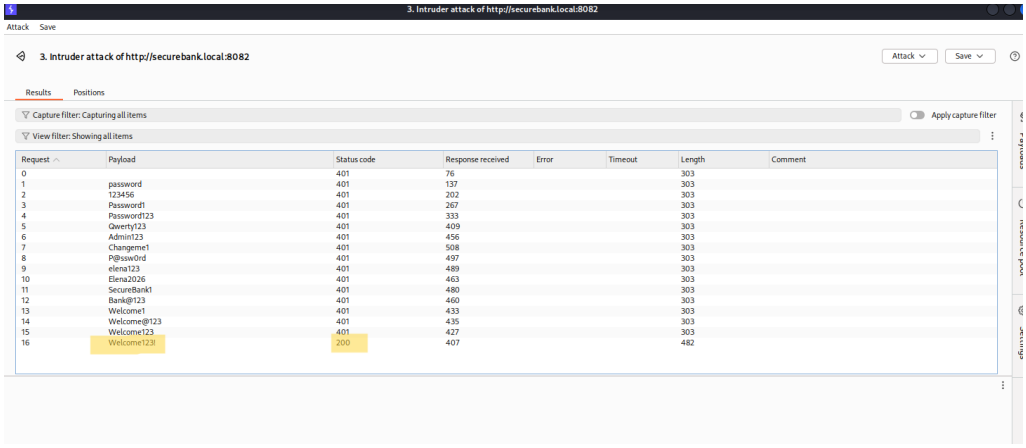


Figure 12 — No rate limiting: Welcome123! returns HTTP 200 among 401 responses.

VUL-004 — Stored Cross-Site Scripting (support messages)

Field	Detail
Finding ID	VUL-004
Affected component	POST /api/support (message field), rendered in the support inbox
OWASP	A03:2021 Injection (Cross-Site Scripting)
CWE	CWE-79 Improper Neutralization of Input During Web Page Generation
Validation status	Confirmed — exploited

The support feature lets a customer send a message to the bank (POST /api/support). These messages are stored and later displayed in the “Your conversations” area and to support staff who read them. The author found that the message content is stored and rendered without output encoding, so any HTML or JavaScript in a message is interpreted by the browser rather than shown as text. Submitting a message containing an italic HTML tag around the word xsstest caused the text to render in italics, confirming HTML injection. Submitting an image tag with an onerror handler, and a script tag, caused JavaScript to run: a console log of XSS-TEST-001 appeared in the browser console and an alert dialog was produced. Because the message is stored and later viewed by others, this is stored (persistent) cross-site scripting: the script will execute in the browser of any user, including support staff, who opens the conversation.

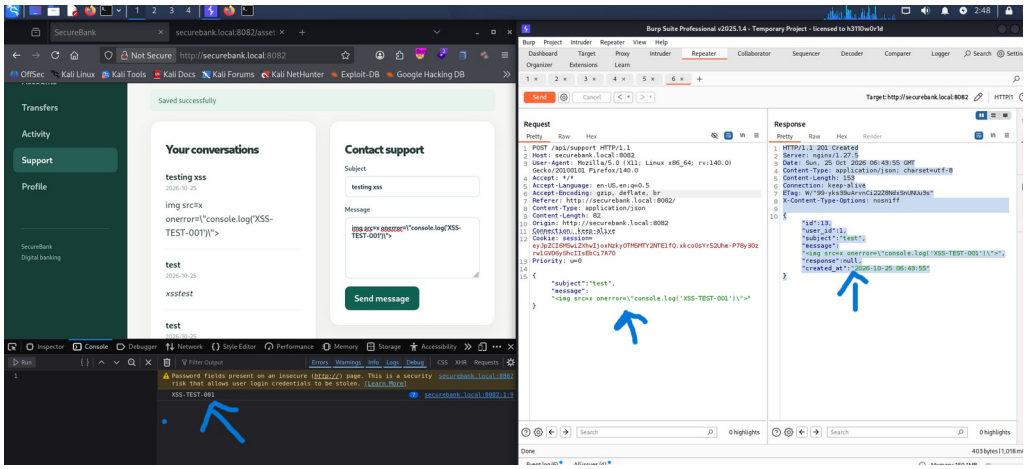


Figure 13 — Stored XSS: image/onerror payload executes (console log).

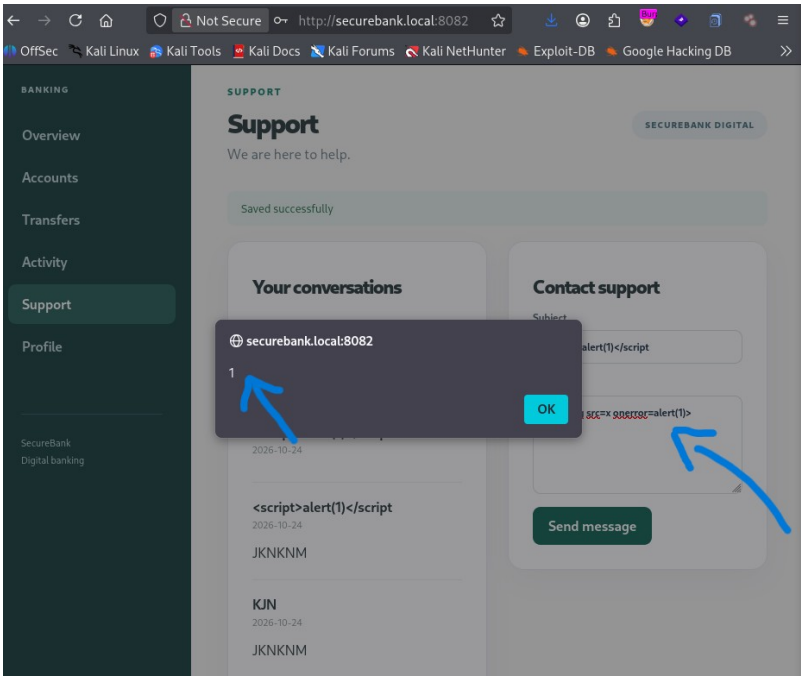


Figure 14 — Stored XSS: script payload raises an alert dialog.

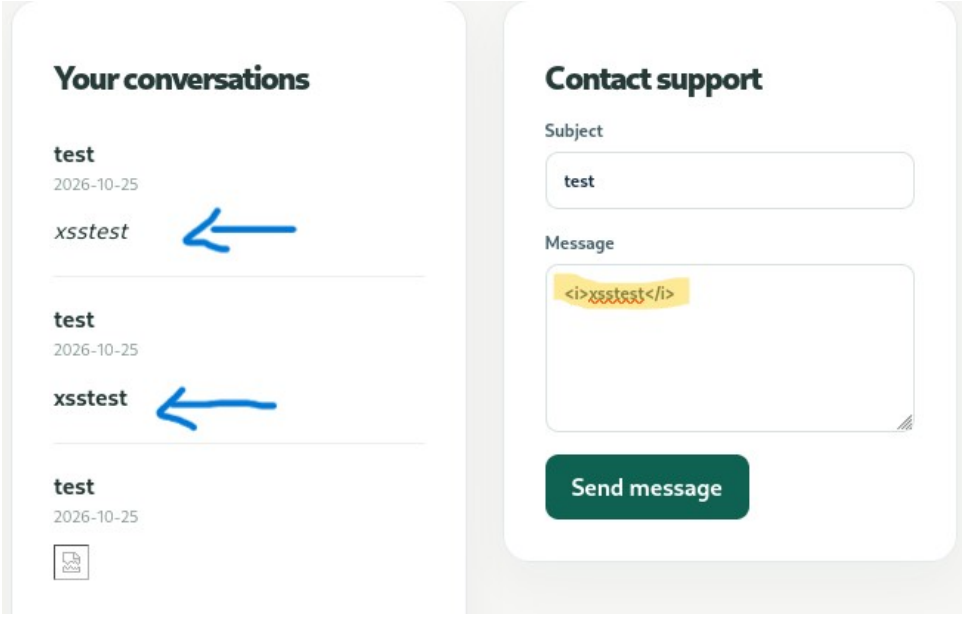


Figure 15 — HTML injection: an italic tag renders as formatted text.

VUL-005 — Security misconfiguration (version disclosure, missing headers, client-side gating)

Field	Detail
Finding ID	VUL-005
Affected component	nginx responses / HTTP security headers / React bundle
OWASP	A05:2021 Security Misconfiguration
CWE	CWE-16 Configuration; CWE-200 Information Exposure; CWE-693 Protection

Field	Detail
	Mechanism Failure
Validation status	Confirmed — observed

Several configuration weaknesses were observed. The server advertises its software and version in the Server response header (nginx/1.27.5), which both discloses information and indicates an out-of-date web server. Scanning with nikto confirmed the outdated server and reported that common security response headers are missing, including Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options (frame-ancestors), Referrer-Policy and Permissions-Policy. The application is also served over plain HTTP rather than HTTPS, so credentials and session tokens are transmitted without encryption. Finally, as established in VUL-001, access control for privileged features is enforced only in the client-side bundle. Individually these issues are low severity, but together they reduce the application's defence-in-depth and make the other vulnerabilities easier to exploit.

Note: nikto additionally reported a JAMon Java admin interface and shell-history files; these are inconsistent with a React/Go/nginx stack and are treated as unconfirmed, likely false positives, pending manual verification.

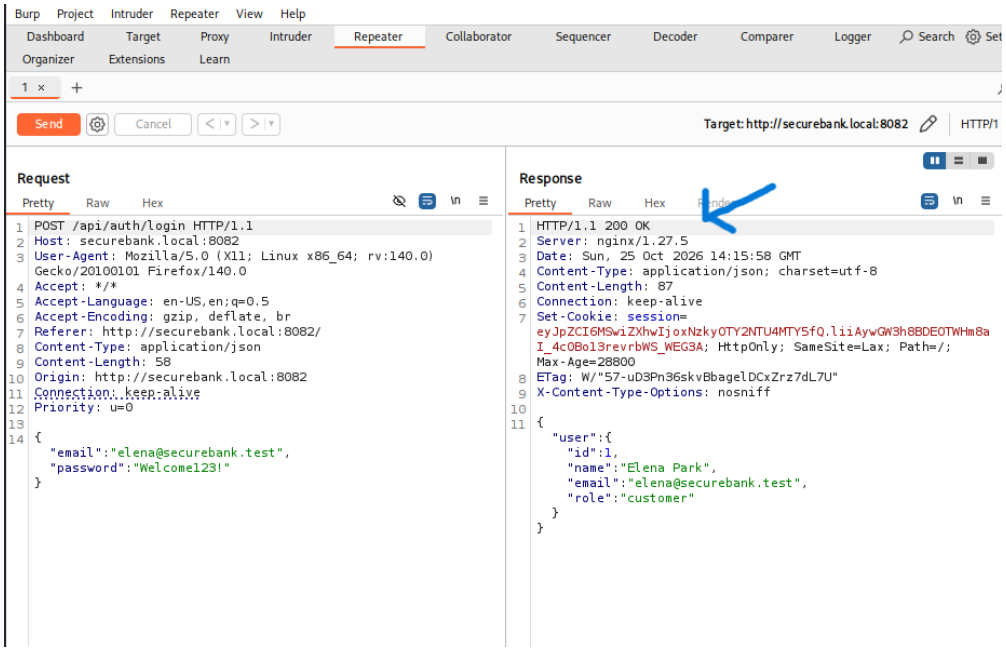


Figure 16 — Information disclosure: Server: nginx/1.27.5 response header.

VUL-006 — SQL injection on login (tested, not confirmed)

Field	Detail
Finding ID	VUL-006
Affected component	POST /api/auth/login (email, password)
OWASP	A03:2021 Injection
CWE	CWE-89 SQL Injection
Validation status	Unverified — tested, no injection confirmed

Because injection is a critical vulnerability class, the author tested the login endpoint for SQL injection using sqlmap against the email and password parameters of POST /api/auth/login, at level 3 and risk 2 (Figure 17). sqlmap's heuristic checks indicated that the JSON email parameter was likely not injectable, and the run did not confirm an exploitable SQL injection point. This finding is therefore recorded as tested but not confirmed: no SQL injection vulnerability was demonstrated. It is included to document that the class was examined and to distinguish a tested-and-negative result from an untested area.

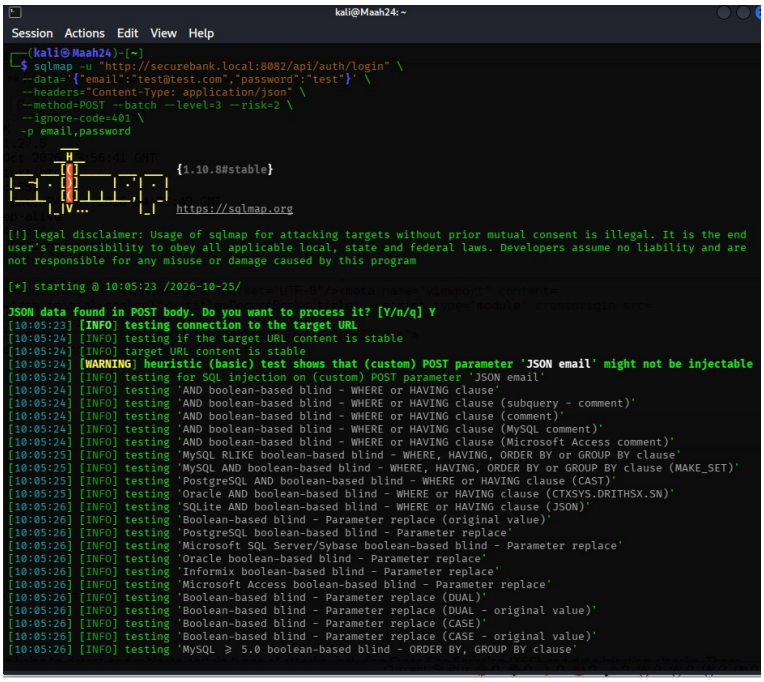


Figure 17 — sqlmap test of the login: no SQL injection confirmed.

B6. Exploitation and Proof of Concept

Controlled exploitation of the four confirmed, exploitable findings is described below. Each refers to the evidence figures shown in Section B5. All exploitation was performed within the authorised test environment.

VUL-001 — Broken function-level access control

Objective: to retrieve all customers' account data using only an ordinary customer session. Logged in as a standard customer, the author issued GET /api/operations in Burp Repeater. The response (Figure 7) returned HTTP 200 and a complete list of customer accounts and balances belonging to many different customers. The same data was retrieved using curl with the session cookie (Figure 8), confirming the finding is reproducible outside the browser. No administrative privilege, and no interaction from any other user, was required.

VUL-002 — IDOR / Broken Object-Level Authorisation

Objective: to read an account belonging to another customer. Using Burp Repeater and a valid customer session, the author requested /api/accounts/2 and /api/accounts/3 (Figures 9 and 10). Each returned HTTP 200 with the account's number, type and balance, and the user_id field showed the records belonged to different customers. By changing a single number in the URL, the author moved from their own data to another customer's data, with no additional privilege.

VUL-003 — Broken authentication (brute force)

Objective: to recover a customer's password by brute force. The author configured a Burp Intruder Sniper attack with the password field as the payload position and a list of sixteen candidate passwords (Figure 11). All fifteen incorrect guesses returned 401 with the same response length, while the sixteenth, Welcome123!, returned 200 with a different length (Figure 12), clearly identifying the valid credential. The attack completed without any rate limiting, lockout or challenge.

VUL-004 — Stored Cross-Site Scripting

Objective: to store script that executes when the message is viewed. The author submitted support messages containing an image/onerror payload and a script-tag alert payload (Figures 13 and 14). On viewing the conversation, the injected JavaScript executed, logging to the console and raising an alert, and a separate italic-tag payload rendered as formatted HTML (Figure 15). This confirms that attacker-controlled script is stored and executed in the viewer's browser, which in a staff context could be used to hijack a session or act as that user.

B7. Post-Exploitation and Attack-Path Analysis

The confirmed vulnerabilities do not exist in isolation; they combine into a realistic attack path, shown in Figure 18. An attacker who knows a customer's email address can brute-force the login (VUL-003), because there is no rate limiting, and obtain a valid customer session. From that ordinary, low-privilege session the attacker can call the staff-only operations endpoint (VUL-001) and can also iterate through account identifiers (VUL-002), reading the names, account numbers and balances of every customer. The effective result is that a low-privilege customer account grants access to data that should be restricted to staff — a failure of authorisation rather than a conventional escalation to an administrator role.

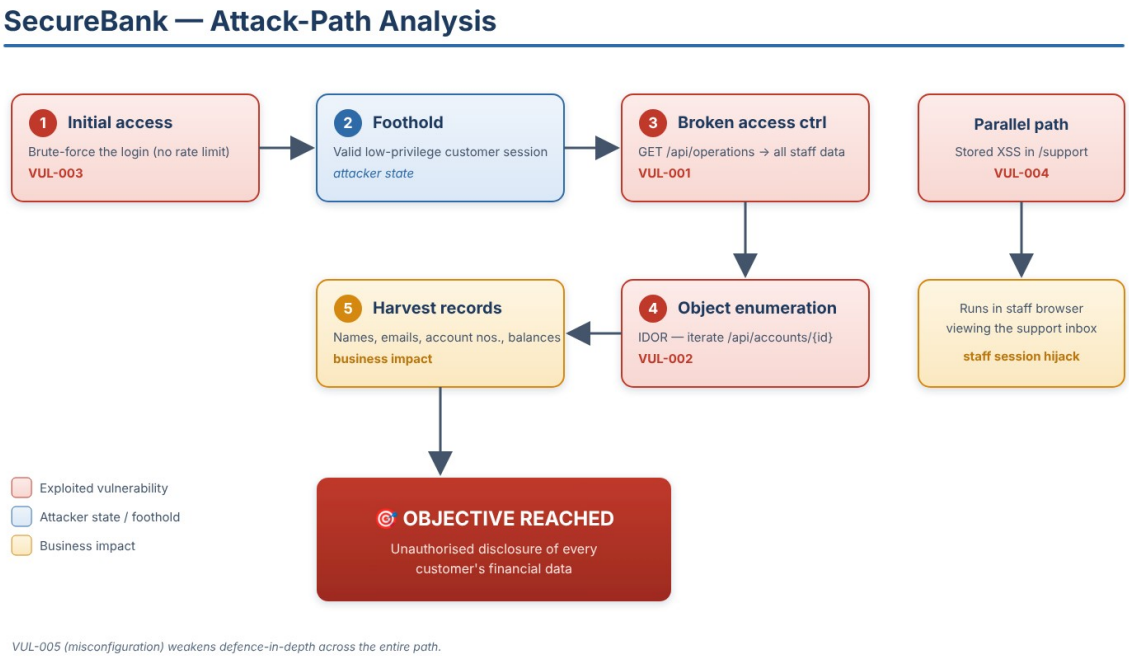


Figure 18 — Attack path from initial access to disclosure of all customer data, with the parallel stored-XSS route against staff.

A separate, parallel path exists through the support feature: a stored cross-site scripting payload (VUL-004) placed in a support message will execute in the browser of a staff member who opens the conversation, and could be used to steal that user's session or act as them. This is the most likely route to compromising a genuinely privileged account.

It is important to state what was not demonstrated. The author did not obtain an administrative account, nor escalate to the underlying server or database; testing remained within the application. This is because the application does not rely on a separate admin role to protect the operations data: the data is simply exposed to any authenticated user, so no escalation was required to reach it. Where a true staff role would be needed (for example, to act within the operations interface itself), the realistic route would be the stored-XSS path against a staff viewer rather than a direct escalation. No post-exploitation evidence was fabricated, and the trust boundary crossed is the application's own authorisation layer.

B8. Vulnerability Risk Assessment

Each confirmed finding has been assigned a severity using CVSS v3.1; the vectors, scores and reasoning are shown below. The scores reflect technical severity under the stated conditions. CVSS and business risk are not identical: VUL-001 and VUL-002 score as Medium under CVSS (because they require a low-privilege account and affect confidentiality only), yet for a bank the business impact of disclosing all customers' financial data is High. The risk table should therefore be read alongside the business-impact discussion that follows.

ID	CVSS v3.1 vector	Score	Severity	Reasoning
VUL-003	`AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`	**7.4**	**High**	Network, unauthenticated; depends on a guessable password (AC:H). Leads to full account takeover (C:H/I:H).
VUL-001	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`	**6.5**	**Medium**	Needs only a low-privilege account; discloses all customers' financial data (C:H). CVSS Medium, business impact High.
VUL-002	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`	**6.5**	**Medium**	Authenticated customer reads another customer's sensitive account data (C:H).
VUL-004	`AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N`	**5.4**	**Medium**	Stored script runs in a victim's browser (scope change); victim must view the message (UI:R).
VUL-005	`AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`	**5.3**	**Medium**	Information exposure / weak defaults; its real value is enabling other attacks.
VUL-006	Not scored	—	N/A	Unverified — no confirmed vulnerability to score.

Viva note: *if VUL-003 is framed as pure brute-force enablement, AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N gives 9.1 (Critical); the AC:H choice above reflects dependence on a guessable password and should be justified accordingly.*

B9. Business Impact Analysis

For SecureBank Ltd, the confirmed weaknesses translate into concrete business consequences. The broken access control and IDOR findings (VUL-001, VUL-002) allow the confidentiality of every customer's financial data to be breached using a single customer account; for a bank this is among the most damaging outcomes possible, exposing names, account numbers and balances. The authentication weakness (VUL-003) allows individual accounts to be taken over through password guessing, opening the door to unauthorised access and, potentially, fraudulent transactions. The stored XSS finding (VUL-004) puts support staff at risk and could be used to compromise more privileged accounts. The configuration weaknesses (VUL-005), including unencrypted HTTP, increase the likelihood that credentials and data are intercepted and weaken the application's overall resilience.

Beyond the direct technical impact, these issues carry regulatory and reputational consequences. Unauthorised disclosure of customers' personal and financial data would breach the UK GDPR and the Data Protection Act 2018, potentially attracting regulatory action, and would seriously damage customer trust in a bank whose core promise is to keep money and information safe. The likelihood of exploitation is high, because the weaknesses require only an ordinary account and widely available tools.

B10. Remediation Recommendations

The following remediations address the root cause of each finding rather than its symptoms; priorities are shown in the table. The single most important change is to enforce authorisation on the server for every request: for each API call the server must check that the authenticated user is permitted to use the specific function and to access the specific object requested, and must deny by default. Hiding features in the front-end must never be relied upon as a control.

For the login, anti-automation controls (rate limiting, temporary lockout or back-off, and a challenge after repeated failures) should be added, together with a strong password policy, the removal of default passwords, and ideally multi-factor authentication. User-supplied content such as support messages must be contextually output-encoded when displayed, with a Content-Security-Policy as defence-in-depth. The server configuration should be corrected by suppressing the version header, adding the missing security headers, updating nginx and, critically, serving the application only over HTTPS/TLS. After remediation, each finding should be retested to confirm the fix, and access-control tests should be added to the development pipeline to prevent regressions.

ID	Root cause	Recommended fix	Priority
VUL-001	Operations endpoint performs no server-side authorisation	Enforce role-based authorisation on the server for /api/operations; deny by default; return 403 to non-staff; never rely on hidden UI menus	High
VUL-002	Object ownership not checked before returning the record	Verify the requested account's owner equals the authenticated user before responding; add automated access-control tests	High
VUL-003	No anti-automation on the login endpoint	Add rate limiting + temporary lockout/back-off; CAPTCHA after N failures; strong password policy; remove default passwords; consider MFA; log and alert on brute force	High
VUL-004	User input rendered without output encoding	Apply contextual output encoding/escaping; rely on framework auto-escaping; sanitise any rich text (e.g. DOMPurify); add a Content-Security-Policy	High

ID	Root cause	Recommended fix	Priority
VUL-005	Insecure defaults and client-side-only controls	server_tokens off; add the missing security headers; update nginx; move authorisation server-side; serve over HTTPS/TLS	Medium

B11. Conclusion

This engagement assessed the security of SecureBank Ltd's online banking application and found that, while some controls are implemented correctly (for example, HttpOnly session cookies), the application has serious weaknesses that would allow an attacker to breach the confidentiality of all customer data and to take over customer accounts. Five vulnerabilities were confirmed and demonstrated, and a sixth area was tested and found not to be exploitable. The engagement objectives were met: the attack surface was mapped; vulnerabilities were identified, validated and exploited within the authorised environment; their business impact was assessed; and remediations were recommended.

The common thread across the most serious findings is a single design flaw: the application decides who may do what in the browser, but does not enforce those decisions on the server. Correcting this — by enforcing authorisation server-side for every function and every object, together with hardening the login and the server configuration — would remove the most damaging risks identified in this report.

References

Cite these in the text and format them in the style your module requires (e.g. Harvard or IEEE).

- OWASP (2021) OWASP Top 10:2021. https://owasp.org/Top10/
- OWASP (2023) OWASP API Security Top 10:2023. https://owasp.org/API-Security/
- OWASP Web Security Testing Guide (WSTG). https://owasp.org/www-project-web-security-testing-guide/
- OWASP Application Security Verification Standard (ASVS). https://owasp.org/www-project-application-security-verification-standard/
- MITRE CWE: CWE-862, CWE-639, CWE-307, CWE-79, CWE-16, CWE-200, CWE-693. https://cwe.mitre.org/
- NIST (2008) SP 800-115: Technical Guide to Information Security Testing and Assessment.
- FIRST (2019) CVSS v3.1 Specification Document. https://www.first.org/cvss/
- Penetration Testing Execution Standard (PTES). http://www.pentest-standard.org/
- MITRE ATT&CK. https://attack.mitre.org/

Appendix A — Evidence Index

Screenshot file	Finding	What it shows
nmap.png	Recon	Port/service scan: nginx 1.27.5 on 8082, Go backend on 36279
nikto.png	Recon / VUL-005	Web-server scan: outdated nginx, missing security headers (and some likely false positives)
Found javascriptfile.png	Recon	Located the React bundle via the index page (/assets/index-*.js)
renamed bankbundle.js.png	Recon	Downloaded the bundle as bank_bundle.js for analysis
Hidden api endpoint.png	VUL-001	grep of the bundle revealed roles and the /operations route gated client-side
Operations endpoint exposed.png	VUL-001	Burp: GET /api/operations returns all accounts (200)
exposed endpoint using curl.png	VUL-001	curl with a customer session dumps all accounts + messages; /api/admin/users = 404
IDOR - 1.png	VUL-002	GET /api/accounts/2 (user_id 1)
IDOR -2.png	VUL-002	GET /api/accounts/3 (user_id 2 — different customer)
Bruteforce attack.png	VUL-003	Burp Intruder setup on /api/auth/login
No rate limit - BF Attack.png	VUL-003	16 attempts, no blocking; Welcome123! returns 200
Stored XSS.png	VUL-004	img onerror payload stored; console.log('XSS-TEST-001') executed
Stored XSS 2.png	VUL-004	<script>alert(1)</script> / img onerror → alert popup
XSS - Bold Italic.png	VUL-004	<i> tag rendered — confirms unsanitised HTML
Info disclosure - Server version.png	VUL-005	Response header Server: nginx/1.27.5
SQL Injection testing.png	VUL-006	sqlmap run on login — no injection confirmed

Appendix B — Tools Used

Tool	Purpose in this engagement
nmap	Service / port discovery
nikto	Web-server misconfiguration scan
Burp Suite	Interception, Repeater (manual testing), Intruder (brute force)
curl	Raw HTTP requests and endpoint checks
sqlmap	SQL-injection testing (login) — no injection confirmed
Firefox DevTools	Traffic, cookies, console (XSS confirmation)
grep	Analysing the JavaScript bundle for routes / roles